Power Spectral Density of Line Code

TEAM 01

Youssef Mohammed Ibrahim Sayed

Youssef Hisham Ahmed Hikel

Mina Wael Tanagho Fahmy

Nader Hany Maurice Wahib

Abdelrahaman Ahmed Kamal Atta

Faculty of Engineering
Department of Electronics and Communications
Course code : EECG232-17
Course name : Introduction to Communication Systems

Power Spectral Density of Line Code

TEAM 01

يوسف محمد ابراهيم سيد

Student ID : 91240892
Section : 4
BN : 34

يوسف هشام أحمد هيكل

Student ID : 91240907
Section : 4
BN : 36

مينا وائل تناغو فهمي

Student ID : 91240804
Section : 4
BN : 14

نادر هاني موريس وهيب

Student ID : 91240805
Section : 4
BN : 15

عبدالرحمن أحمد كمال عطا

Student ID : 91240410
Section : 2
BN : 24

Supervisor: Dr. Mohammed Nafie
Professor, Cairo University

Co-supervisor: ENG. Mohammed Khaled
Teacher Assistant, Cairo University

Faculty of Engineering

Department of Electronics and Communications

Course code : EECG232-17

Course name : Introduction to Communication Systems

Role of each member

The project was divided into specialized roles to ensure a comprehensive analysis of stochastic processes and digital signaling. The responsibilities were distributed as follows:

- **Youssef Mohammed Ibrahim**:
  - Performed the hand analysis and rigorous mathematical derivations for the mean and autocorrelation functions of the line codes.
  - Helped in creating the autocorrelation function and computing the power spectral density (PSD) for each signaling scheme.
  - Helped in the report formatting and organization.
- **Youssef Hisham Ahmed**:
  - Hepled in the generating the ensemble of realizatons for each line code and the generation of data.
  - Managed the simulation environment settings, ensuring proper sampling frequencies and time-domain parameters were maintained across all models.
- **Nader Hany Maurice**:
  - Conducted the verification of Wide-Sense Stationarity (WSS) by analyzing the ensemble mean and autocorrelation across varying time shifts.
  - Implemented the ergodicity tests, comparing time-averages of individual realizations against the statistical ensemble averages.
- **Mina Wael Tanagho**:
  - Analyzed the Power Spectral Density plots to identify key spectral features, including main-lobe bandwidth and the presence of discrete spectral components.
  - Verified the consistency between simulated PSD envelopes and the theoretical models.
- **Abdelrahman Ahmed Kamal**:
  - Validated the simulated waveform behavior against the theoretical pulse definitions for NRZ and RZ schemes.
  - Helped in the report formatting and organization.

Abstract

This project explores the implementation of digital communication systems using the software radio technique. By shifting traditional hardware-based radio problems into the software domain, we simulate a transmitter and receiver environment that closely mirrors real-world antenna behavior. The primary focus is the generation and analysis of three specific line codes: Polar RZ, Polar NRZ, and Unipolar.

Since the transmitted bit-streams and initial timing offsets are inherently random, the system is modeled as a random process. Using MATLAB, we generate an ensemble of realizations for each signaling technique to evaluate their statistical and time-domain characteristics. Specifically, we analyze the statistical mean, time mean, autocorrelation functions, and power spectral density. These results are then used to verify the stationarity and ergodicity of the processes, providing a comprehensive understanding of how software-defined waveforms behave in a practical communications environment.

Contents

1 Problem Description **2**

2 Introduction **3**

3 Control Flags **4**

4 Generation of Data **5**

5 Creating the Unipolar Ensemble **6**

6 Creating The Polar NRZ Ensemble **8**

7 Creating The Polar RZ Ensemble **9**

8 Q1 Statistical Mean Calculation **10**

9 Q2 Is the Process Stationary **13**

10 Q3 Statistical Autocorrelation + Q4 Time Mean & Autocorrelation of One Waveform **16**

11 Q5 Is the Random Process Ergodic **19**

12 Plotting the PSD of the Ensemble **20**

13 Q6 Signal Bandwidth **22**

Appendices

A Appendix **25**

References 32

1

Problem Description

In this project, we are simulating a software radio. Software radio is a technique that uses software to perform radio signal processing tasks that were traditionally done by hardware. The software radio allows for greater flexibility and adaptability in radio communication systems, as it can be easily reconfigured and updated through software changes. In this project, we will be simulating a software radio using MATLAB. We will be implementing various components of the software radio, such as modulation, demodulation, filtering, and channel coding, to create a complete simulation of a software radio system and see the problems ourselves and try design a proper solution.

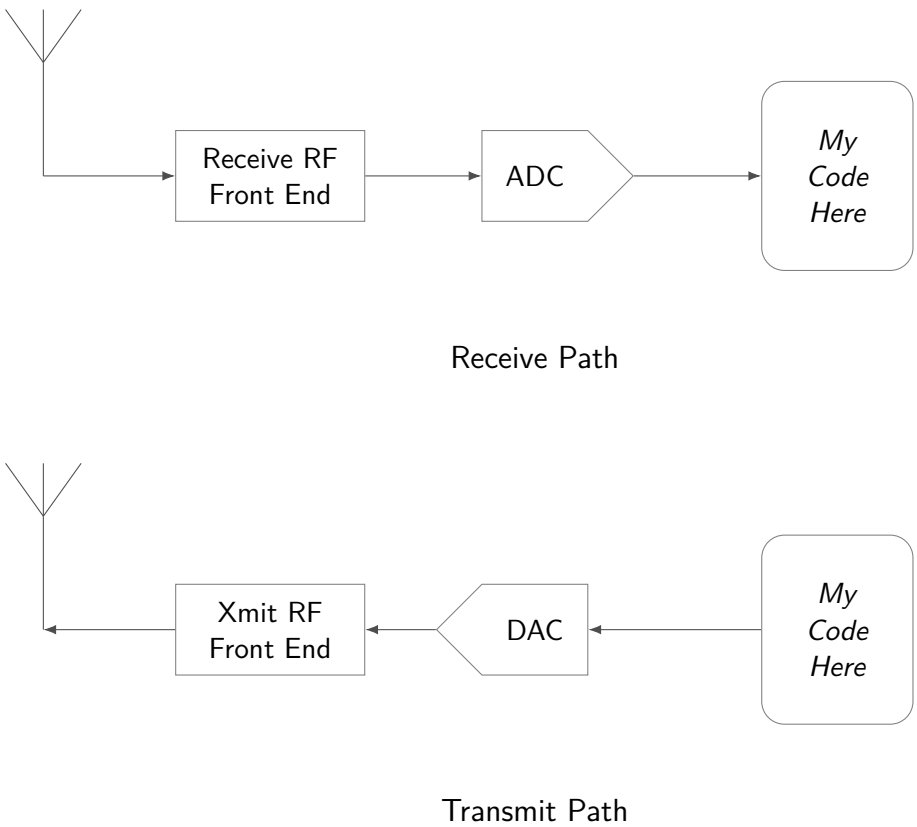

Figure 1.1: *Software Radio Transmit and Receive Paths.*

2

Introduction

In this project, we are going to use the software radio technique to simulate the hardware problems and try to make it as close as it is with the antenna, while also using the techniques we learned through our communications course. We are going to design the transmitter first. Our code at the transmitter will define the modulation technique and the coding used. Once the digital data is generated, it goes through a Digital-to-Analog Converter (DAC) so it can actually take the form of three different transmitted signals: Polar RZ, Polar NRZ, and Unipolar. However, the bit-stream generated at the transmitter is actually random, and there can also be asynchronization between the transmitter and the receiver, which is also random. Therefore, in our simulation, we will generate a random bit-stream, and the output will be treated as a random process. Then, for each generated ensemble, we will calculate its statistical mean, the time mean, the ensemble and time autocorrelation functions, and the bandwidth. We will also determine whether the random process is stationary and ergodic.

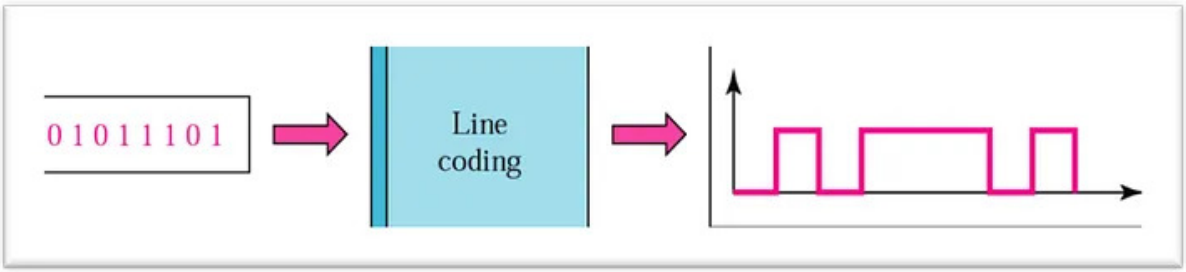

Figure 2.1: *A simple illustration of the line coding process.*

3

Control Flags

Listing 1: *Defining the control flags.*

```
%% Control Flags
N_bits         = 100;           % Number of bits per realization.
N_realizations = 500;           % Number of realizations.
Pw             = 0.07;          % Pulse width of the bit.
Ts             = 0.01;          % Time sample.
L              = round(Pw/Ts);  % Samples per symbol period (L=7).
A              = 4;             % Amplitude.
N_fft          = 1024;          % FFT size.
Fs             = 1/Ts;          % Sampling frequency.
freq           = (-N_fft/2:N_fft/2-1) * (Fs/N_fft);  % Frequency axis.
center         = floor(N_fft/2) + 1;                  % Center bin.
half_len       = 100;           % Max tau used in correlation_manual.
```

- `N_bits`: Defines the number of bits generated in each realization of the random process. It controls the length of the bit stream.
- `N_realizations`: Specifies the number of independent realizations of the random process.
- `Pw`: Represents the pulse width (bit duration) in seconds. It determines the time-domain duration of each bit and directly affects the signal's bandwidth.
- `Ts`: The time sampling interval in seconds.
- `L`: It determines how many discrete samples represent a single bit.
- `A`: Sets the peak amplitude of the transmitted signal.
- `N_fft`: Specifies the size of the Fast Fourier Transform (FFT).
- `Fs`: The sampling frequency $(1/Ts)$. It defines the frequency range of the simulation and must be high enough to satisfy the Nyquist criterion.
- `freq`: The frequency axis vector used for spectral analysis, mapped from $-Fs/2$ to $Fs/2$ to allow for centered Power Spectral Density (PSD) plots.
- `center`: The index of the zero-frequency (DC) component in the FFT output, used to correctly align and shift the spectrum for plotting.
- `half_len`: Defines the maximum lag (τ) for the manual calculation of the autocorrelation function.

4

Generation of Data

Now that the control flags are defined, we can proceed with the first step of the simulation: generating the raw binary data. To accurately simulate a real-world communication source, we use the MATLAB function `randi` to produce a random bit-stream.

Listing 2: *Generation of Data.*

```
%% Data Generation
data = randi([0 1], N_realizations, N_bits);
```

The function `randi([0 1],` $N_{\texttt{realizations}}$`,` $N_{\texttt{bits}}$`)` generates a matrix of independent random integers. In this setup, each row represents a unique *realization* of the random process, while each column represents a *bit* in the sequence.

This matrix structure is fundamental to our analysis; it allows us to treat the entire project as a discrete ensemble. By performing operations across the rows, we can calculate ensemble statistics (like the statistical mean), and by operating across the columns, we can determine time-domain characteristics (like the time average). Furthermore, using a range of $[0, 1]$ ensures an equiprobable source where the probability of a "0" or a "1" is exactly 0.5, providing a neutral baseline for comparing the efficiency of different line codes.

Realizations

Realization 1	1	0	1	0	1	1	0	1	0	1	0	0	0	0	1	0	1	0	0	1	0	1	1	0
Realization 2	1	0	0	1	0	1	0	1	0	1	0	1	1	1	1	1	1	0	0	0	0	0	0	0
Realization 3	0	1	1	0	1	0	1	0	1	0	0	0	0	0	0	1	1	1	1	0	1	0	1	0
Realization 5	1	0	0	1	0	1	1	1	1	1	1	1	0	0	0	1	1	1	0	0	1	1	1	1
Realization 1000	0	1	0	1	0	0	1	1	1	1	0	0	0	0	0	0	1	1	1	0	1	1	0	0
Realization 9999	1	0	1	0	0	0	0	1	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0
Realization 10^9	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Realization ∞	1	1	0	1	0	0	0	1	0	1	0	0	1	1	1	1	1	1	1	1	1	0	0	0

t

Figure 4.1: *A visual example of the generated data matrix.*

5

Creating the Unipolar Ensemble

Now that we have defined the control flags and generated the data, we can proceed to create our signals, we will start with the unipolar line code, which might be the simplest among the three line codes we are going to analyze. But first, lets create the ensemble. As we are going to analyze multiple codes we are going to create a function that will create the ensemble then use it to create the ensemble for each code.

Listing 3: *Creating Ensemble function.*

```
function ensample = build_ensemble(data, A, pulse, N_realizations, L, type)
if strcmp(type, 'unipolar'), Tx = data * A;          % Map bits to {0, A}.
else,                        Tx = (2*data - 1) * A;  % Map bits to {-A, +A}.
end
ensample = kron(Tx, pulse);                          % Upsample with pulse shape.
Td = randi([0, L-1], N_realizations, 1);             % Random delay per realization.
for i = 1:N_realizations
    ensample(i,:) = circshift(ensample(i,:), [0, Td(i)]);
end
end
```

The function `build_ensemble` takes the raw binary data and transforms it into a matrix of time-domain signals, where each row corresponds to a different realization of the random process. Also a random time delay is added using `randi` function, then `circshift` is used to apply the random delay to each realization, ensuring that each signal starts at a different time instant.

- `data` parameter is the generated random bits from Listing 3. These data will be mapped to 0 and A for unipolar, and to $-A$ and $+A$ for polar.
- `pulse` parameter defines the shape of the pulse used for upsampling, which in our case is a rectangular pulse of width L.
- `N_realizations` specifies how many independent signal realizations we want to generate, which corresponds to the number of rows in the output matrix.
- `L` is the number of samples per symbol period, which determines how the bits are upsampled in time.
- `type` is a string parameter that specifies the line code type, allowing the function to handle both unipolar and polar mappings and distinguish between them when generating the ensemble.

Now that we created the function to create the ensemble, we can use it to create the unipolar ensemble that we aimed to create initially in this part of the project.

Listing 4: *Creating the Unipolar Ensemble.*

```
%% Unipolar NRZ
fprintf('\nUnipolar NRZ\n');
pulse_unipolar  = ones(1, L); % Pulse shape for unipolar NRZ (rectangular pulse).
ylim_unipolar   = [-1, A+1]; % Set y-axis limits for the plot.
color_unipolar  = [0.8500 0.3250 0.0980]; % Orange color for the plot.

ensample_unipolar = build_ensemble(data, A, pulse_unipolar, N_realizations, L, 'unipolar');

```

All we did in Listing 4 is setting the pulse parameter and passing all the existing parameters to the `build_ensemble` function, for the data we used the generated data from Listing 3, for `A`, $N_{realizations}$, `L` we used the defined amplitude, number of realizations, and Samples per symbol period from the control flags in Listing 1.

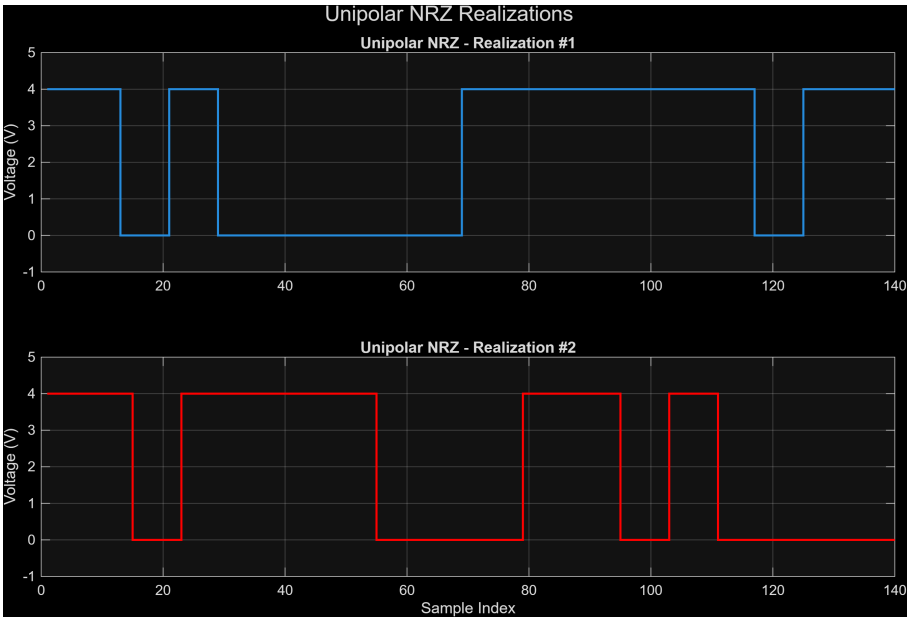

Figure 5.1: *Realization 1 and Realzation 2 from the Unipolar Ensemble.*

Now we can see in Figure 5.1 the first two realizations of the unipolar ensemble, the signal is a rectangular pulse as we expected from the input pulse, and the amplitude maps to 0 and 4 as how we assigned A earlier in the control flags. This output confirms that we succefully created the unipolar ensemble as we met the required specifications for the unipolar line code.

- The amplitude of the signal is either 0 (for bit "0") or A (for bit "1").
- The pulse shape is rectangular, with a duration of L samples per bit.

6

Creating The Polar NRZ Ensemble

Similar to what we did in the previous chapter when we were creating the unipolar ensemble, we will apply the same steps to create the polar NRZ ensemble.

Listing 5: *Creating the Polar NRZ Ensemble.*

```
%% Polar NRZ
fprintf('\nPolar NRZ\n');
pulse_polar_NRZ = ones(1, L);
ylim_polar_NRZ  = [-A-1, A+1];
color_polar_NRZ = [0 0.4470 0.7410];

ensample_polar_NRZ = build_ensemble(data, A, pulse_polar_NRZ, N_realizations, L, 'polar');
```

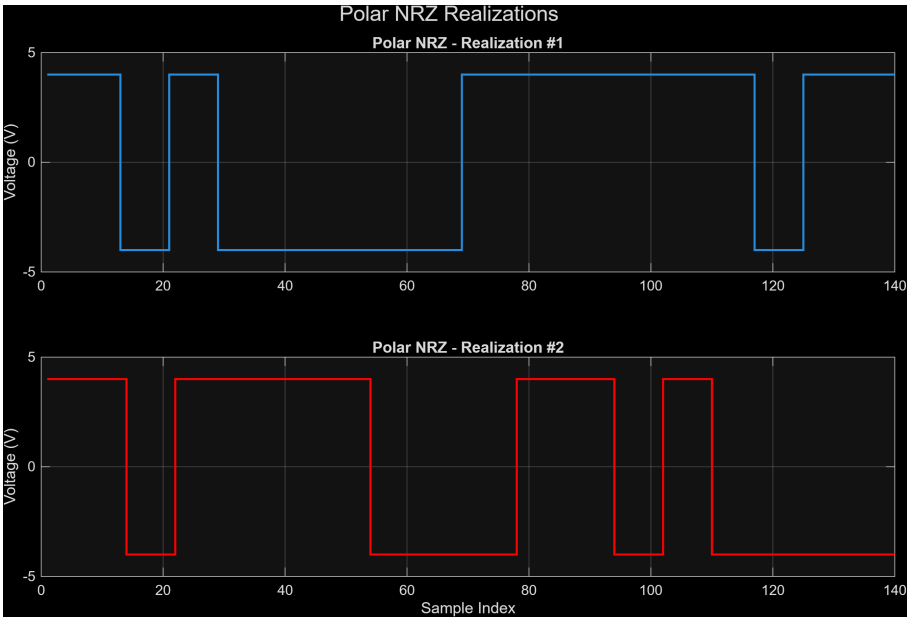

Figure 6.1: *Realization 1 and Realzation 2 from the Polar NRZ Ensemble.*

As we can see in Figure 6.1

- The signal is either at $-A$ or at $+A$, which is the expected behavior for the polar NRZ line code.
- The pulse shape is rectangular, with a duration of L samples per bit.

7

Creating The Polar RZ Ensemble

Nothing changed or to say here, we will just apply the same steps again.

Listing 6: *Creating the Polar RZ Ensemble.*

```
%% Polar RZ
fprintf('\nPolar RZ\n');
pulse_RZ = [ones(1, floor(L/2)), zeros(1, ceil(L/2))];
ylim_RZ  = [-A-1, A+1];
color_RZ = [0.4940 0.1840 0.5560];

ensample_RZ = build_ensemble(data, A, pulse_RZ, N_realizations, L, 'polar');
```

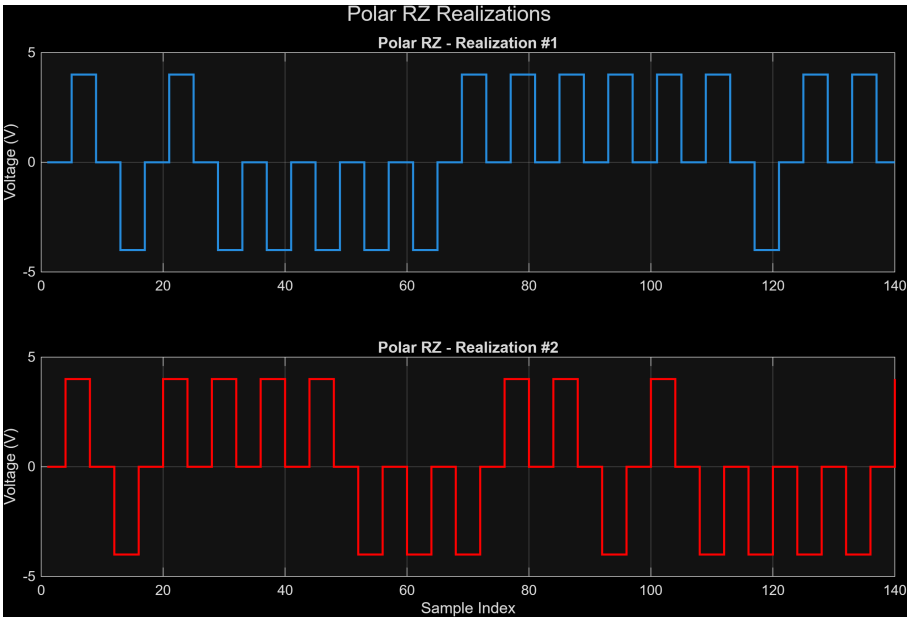

Figure 7.1: *Realization 1 and Realzation 2 from the Polar RZ Ensemble.*

As we can see in Figure 7.1

- The signal is now a pulse that lasts for half of the bit duration, and then goes to zero for the other half. This is the main characteristic of RZ signals.
- The pulse shape is rectangular, with a duration of L samples per bit.

8

Q1 Statistical Mean Calculation

First, we will create a function that can caluclate the mean of the ensemble of any signal we generated.

Listing 7: *Manual Mean Calculation Function.*

```
function mean_val = manual_mean(data, dim)
if nargin == 1,    mean_val = sum(data, 'all') / numel(data);  % All elements.
elseif dim == 1,   mean_val = sum(data, 1) / size(data, 1);    % Column-wise (ensemble).
elseif dim == 2,   mean_val = sum(data, 2) / size(data, 2);    % Row-wise (time).
end
end
```

The function `manual_mean` is designed to compute the mean of any signal passed to it,

The logic of the function is designed as how we were taught in the course on how to calculate the mean specfically in [1]

$$\mu = \sum_{i=1}^{N} p(x_i)x_i \tag{8.1}$$

Equation 8.1 shows the general expression for calculating the mean of a random variable, now since the $p(x_i)$ is always the same for all bits, so we can simplify the equation to be:

$$\mu = \frac{1}{N}\sum_{i=1}^{N} x_i \tag{8.2}$$

In Listing 7 we have implemented Equation 8.2 to calculate the mean of any input to this function, then we adjusted the input parameters to be able to calculate the mean either for the ensemble or for the time samples.

- If the user does not specify the dimenstion, the function will calculate the mean of all the elements in the input data.
- If the user specifies the dimension as 1, the function will calculate the mean column-wise, which is the mean of the ensemble for each time sample.
- If the user specifies the dimension as 2, the function will calculate the mean row-wise, which is the mean of the time samples for each ensemble member.

Now after creating the functio to calculate the mean manually, we will create another function to make the code easier and more readable, its only purpose is to use the `manual_mean` and make the code less messy.

Listing 8: *Mean Calculation Wrapper Function.*

```
        function [Ex, Ux] = compute_means(ensample, theoretical_mean)
Ex = manual_mean(ensample, 1);   % Ensemble mean (across realizations).
Ux = manual_mean(ensample, 2);   % Time mean (across samples).
fprintf('  Sample 10 ensemble mean  = %.4f V (Theoretical: %.2f V)\n',
↪  manual_mean(ensample(:,10)),  theoretical_mean);
fprintf('  Realization 15 time mean = %.4f V (Theoretical: %.2f V)\n',
↪  manual_mean(ensample(15,:)), theoretical_mean);
end
```

The `compute_means` function takes the ensemble as an argument, also we pass the theoretical mean as an argument to this function to compare the output of the ensemble to the theortical mean calculated by the techniques learned in the course. Now after creating the two functions needed to claculate the mean easily and in an organized order, we can use these functions to calculate the mean of each ensemble we created in one line of code.

Listing 9: *Mean Calculation for Unipolar*

```
    [Ex_unipolar, Ux_unipolar] = compute_means(ensample_unipolar, A/2);
```

Listing 10: *Mean Calculation for Polar NRZ*

```
    [Ex_polar_NRZ, Ux_polar_NRZ] = compute_means(ensample_polar_NRZ, 0);
```

Listing 11: *Mean Calculation for Polar RZ*

```
    [Ex_RZ, Ux_RZ] = compute_means(ensample_RZ, 0);
```

Now to calculate the theortical mean for each ensemble across realizations we can use Equation 8.2.

- For the unipolar ensemble, since the bits are represented by A and 0, the mean can be calculated as:
$$\mu_{unipolar} = \frac{1}{2}(A + 0) = \frac{A}{2} = 2V \tag{8.3}$$
- For the Polar NRZ ensemble, since the bits are represented by A and $-A$, the mean can be calculated as:
$$\mu_{polar_NRZ} = \frac{1}{2}(A + (-A)) = 0 \tag{8.4}$$

- For the Polar RZ ensemble, since the bits are represented by A, $-A$ and 0, the mean can be calculated as:

$$\mu_{polar_RZ} = \frac{1}{3}(A + (-A) + 0) = 0 \tag{8.5}$$

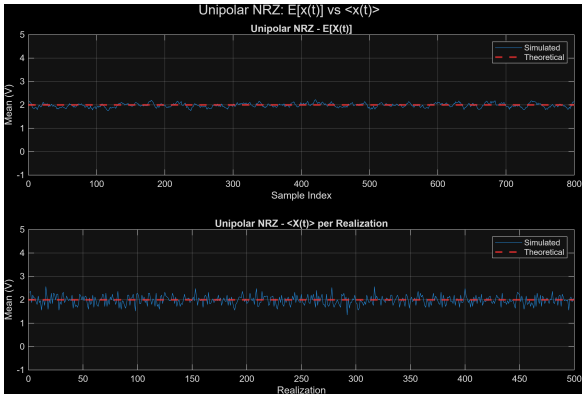

Figure 8.1: *UNIPOLAR NRZ mean across time and across realizations.*

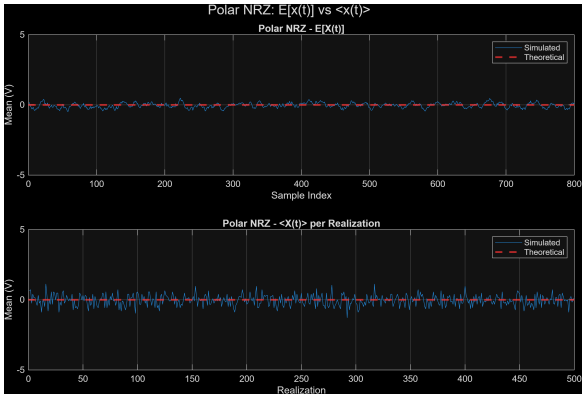

Figure 8.2: *POLAR NRZ mean across time and across realizations.*

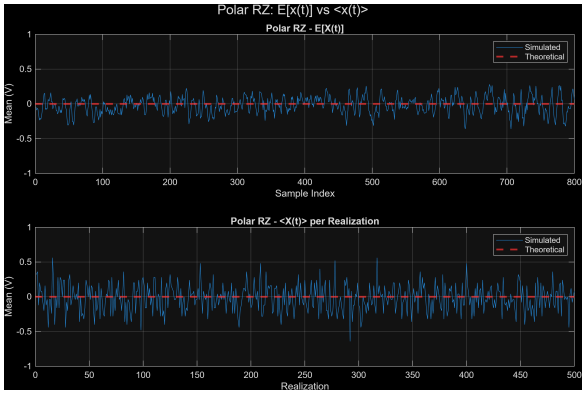

Figure 8.3: *POLAR RZ mean across time and across realizations.*

- In Figure 8.1 we can see that the mean across realizations is $1.7840V$ (Calculated from MATLAB itself) which is nearly the theoretical mean calculated in Equation 8.3
- In Figure 8.2 we can see that the mean across realizations is around $-0.1920V$ which is nearly the theoretical mean calculated in Equation 8.4
- In Figure 8.3 we can see that the mean across realizations is around $-0.2560V$ which is nearly the theoretical mean calculated in Equation 8.5

Note

Graphs in Figure 8.1, Figure 8.2 and Figure 8.3 show both means across time and realizations, in this question we do not need the time mean but we will use it later in the ergodicity check.

9

Q2 Is the Process Stationary

To answe this question, we must know what conditions make a process stationary. As we learned in [2] for a process to be stationary :

1. All random variables must be identically distributed.

$$p(x(t_1)) = p(x(t_2)) = \ldots = p(x(t_n)) \quad \forall t_1, t_2, \ldots, t_n \tag{9.1}$$

2. All joint distributions must depend only on the time differences.

$$p(x(t_1), x(t_2), \ldots, x(t_n)) = p(x(t_1 + \tau), x(t_2 + \tau), \ldots, x(t_n + \tau)) \quad \forall \tau \tag{9.2}$$

Now from simplicity we can use the Wide-Sense Stationarity (WSS) conditions to check if the process is stationary,

1. The mean of the process must be constant over time.

$$E[x(t)] = \mu \quad \forall t \tag{9.3}$$

2. The autocorrelation function must depend only on the time difference.

$$R_x(t_1, t_2) = R_x(t_1 - t_2) \quad \forall t_1, t_2 \tag{9.4}$$

From the previous chapter, we have already calculated the mean of the process across time and it was constant for all signals in equations 8.3, 8.4, 8.5 so condition one is already satisfied. Now to check the second condition, we need to calculate the autocorrelation across different time instances with the same difference and check if it is the same for all time instances. To do this, we will create a function that calculates the autocorrelation. The equation we want to implement in the function is the following equation from [2]:

$$R_x(\tau) = E[x(t)x(t + \tau)] \tag{9.5}$$

Where we get the expectation value of a random variable multiplied by a shifted version of itself. After this question we will use this equation or pretty much the function by getting its `FT` to calculate the power spectral density.

Listing 12: *Autocorrelation Calculation Function*

```
        function [Rx, tau] = correlation_manual(signal, max_tau, type)
tau = -max_tau:max_tau;      % Lag vector.
Rx  = zeros(1, length(tau)); % Output array.
if strcmp(type, 'ensemble')
    [~, N_samples] = size(signal);
    for i = 1:length(tau)
        shift = tau(i);
        if shift >= 0
            Rx(i) = manual_mean(signal(:, 1:N_samples-shift) .* signal(:, 1+shift:N_samples));
        else
            Rx(i) = manual_mean(signal(:, 1-shift:N_samples) .* signal(:, 1:N_samples+shift));
        end
    end
elseif strcmp(type, 'time')
    if isvector(signal), signal = signal(:)'; end  % Ensure row vector.
    N = length(signal);
    for i = 1:length(tau)
        lag = tau(i);
        if lag >= 0
            Rx(i) = manual_mean(signal(1:N-lag) .* signal(1+lag:N));
        else
            Rx(i) = manual_mean(signal(1-lag:N) .* signal(1:N+lag));
        end
    end
end
end
```

This function does exactly what Equation 9.5 says, it calculates the expectation value of the signal, using the `manual_mean` function we created ealier, mulitplied by a shifted version of itself for different values of the shift τ.

Listing 13: *Getting Autocorrelation for Unipolar*

```
        [Rx_unipolar, tau_unipolar]            = correlation_manual(ensample_unipolar, 100, 'ensemble');
        [Rx_time_unipolar, lags_time_unipolar] = correlation_manual(ensample_unipolar(10,:), 100, 'time');
```

Listing 14: *Getting Autocorrelation for Polar NRZ*

```
        [Rx_polar_NRZ, tau_polar_NRZ]            = correlation_manual(ensample_polar_NRZ, 100, 'ensemble');
        [Rx_time_polar_NRZ, lags_time_polar_NRZ] = correlation_manual(ensample_polar_NRZ(10,:), 100, 'time');
```

Listing 15: *Getting Autocorrelation for Polar RZ*

```
[Rx_RZ, tau_RZ]            = correlation_manual(ensample_RZ,       100, 'ensemble');
[Rx_time_RZ, lags_time_RZ] = correlation_manual(ensample_RZ(10,:), 100, 'time');
```

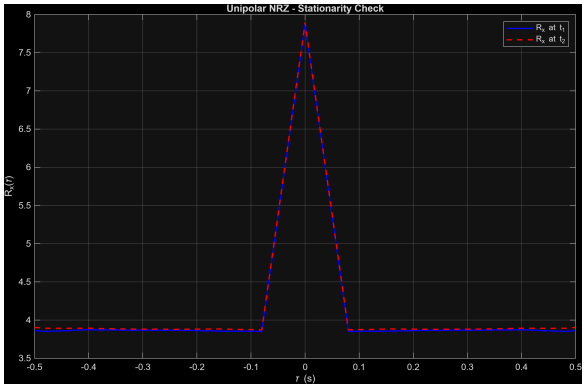

Figure 9.1: *UNIPOLAR NRZ stationarity check.*

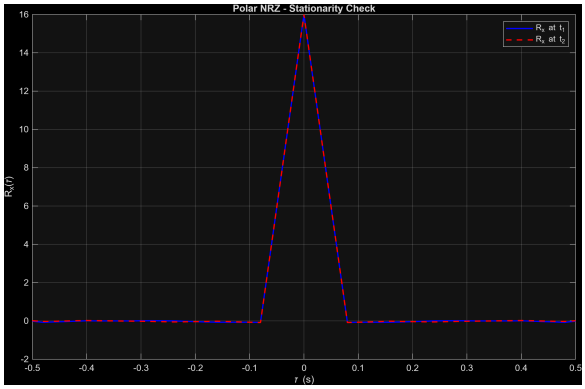

Figure 9.2: *POLAR NRZ stationarity check.*

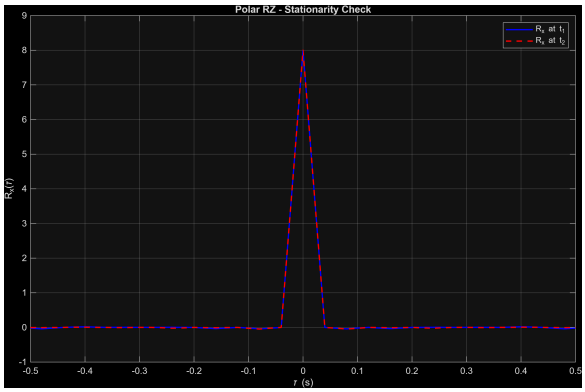

Figure 9.3: *POLAR RZ stationarity check.*

As we can see from the figures above, the autocorrelatoion across time at two different instances is almost or exactly the same as they have the same time difference, therefore we can safely say that all of our three signals are stationary.

10

Q3 Statistical Autocorrelation + Q4 Time Mean & Autocorrelation of One Waveform

In this question, we can use our already established function that we created in the previous question in Listing 12 to calculate the autocorrelation of each signal. For the Polar NRZ signal we can get the autocorrelation by using the following equation from [3] :

$$R_x(\tau) = \frac{A^2}{2}\left(1 - \frac{|\tau|}{T_b}\right) \text{ for } |\tau| \leq T_b \tag{10.1}$$

Also for the Unipolar NRZ signal we can get the autocorrelation by using the following equation from [3] :

$$R(\tau) = \frac{A^2}{4} + \frac{A^2}{4}\Delta\left(\frac{t}{T}\right) \tag{10.2}$$

Now we are going to derive the autocorrelation expression for the Polar RZ signal.

For a Polar RZ signal, symbols are represented by $+A$ and $-A$, but the active pulse only lasts for half the bit duration $(T/2)$, returning to zero for the second half.

- $P(X(t) = A) = P(\text{Bit } 1) \times P(\text{in pulse}) = 0.5 \times 0.5 = 0.25$
- $P(X(t) = -A) = P(\text{Bit } 0) \times P(\text{in pulse}) = 0.5 \times 0.5 = 0.25$
- $P(X(t) = 0) = P(\text{in zero region}) = 0.5$

For $\tau < T/2$:

$$R(\tau) = E[X(t)X(t+\tau)] = \sum X(t)X(t+\tau)P(X(t)X(t+\tau)) \tag{10.3}$$

$$R(\tau) = A^2P(A,A) + (-A)^2P(-A,-A) - A^2P(-A,A) - A^2P(A,-A) \tag{10.4}$$

By symmetry, $P(A,A) = P(-A,-A)$ and $P(-A,A) = P(A,-A)$:

$$R(\tau) = 2A^2P(A,A) - 2A^2P(-A,A) \tag{10.5}$$

Step 1: Finding $P(A,A)$

$$P(A,A) = P(X(t+\tau) = A \mid X(t) = A)P(X(t) = A) \tag{10.6}$$

Given $X(t) = A$, we know we are currently inside an active pulse of width $T/2$. The probability of **not transitioning** out of this pulse is $1 - \frac{\tau}{T/2} = 1 - \frac{2\tau}{T}$. If it does transition, it drops to the 0

region. Thus:

$$P(A, A) = \left(1 - \frac{2\tau}{T}\right) \times 0.25 \tag{10.7}$$

Step 2: Finding $P(-A, A)$

$$P(-A, A) = P(X(t+\tau) = A \mid X(t) = -A)P(X(t) = -A) \tag{10.8}$$

To go from $-A$ to A, the signal must transition to the next bit interval. However, because $\tau < T/2$, the shift τ is too small to cross the $T/2$ zero-region gap between active pulses.

$$P(X(t+\tau) = A \mid X(t) = -A) = 0 \implies P(-A, A) = 0 \tag{10.9}$$

Final $R(\tau)$ **for** $\tau < T/2$**:**

$$\begin{aligned} R(\tau) &= 2A^2 \left[\left(1 - \frac{2\tau}{T}\right) \times 0.25\right] - 0 \\ R(\tau) &= \frac{A^2}{2}\left(1 - \frac{2\tau}{T}\right) \end{aligned} \tag{10.10}$$

For $\tau \geq T/2$, the autocorrelation is 0. This gives us the final triangle function

$$R(\tau) = \frac{A^2}{2}\Delta\left(\frac{2\tau}{T}\right) \tag{10.11}$$

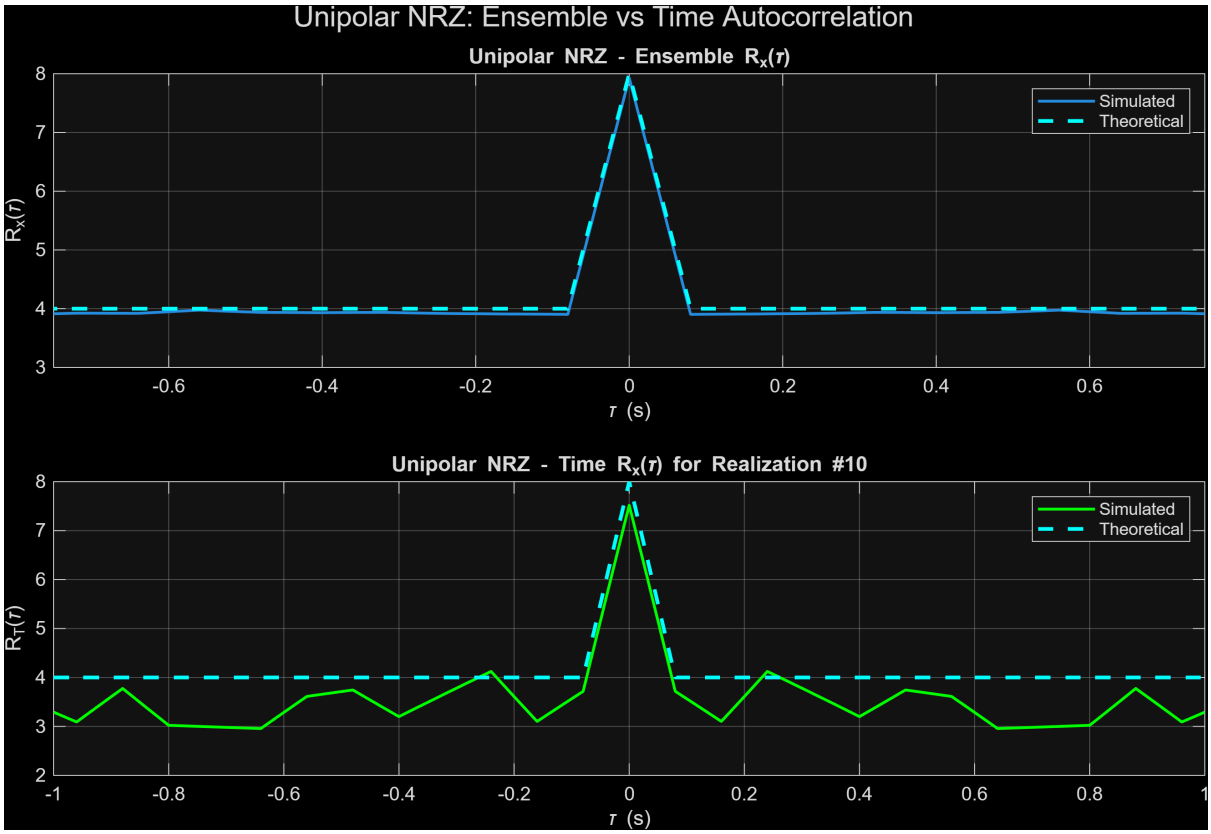

Figure 10.1: *UNIPOLAR NRZ Autocorrelation.*

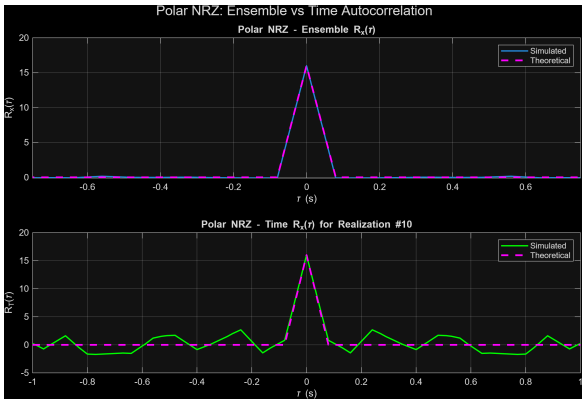

Figure 10.2: *POLAR NRZ Autocorrelation.*

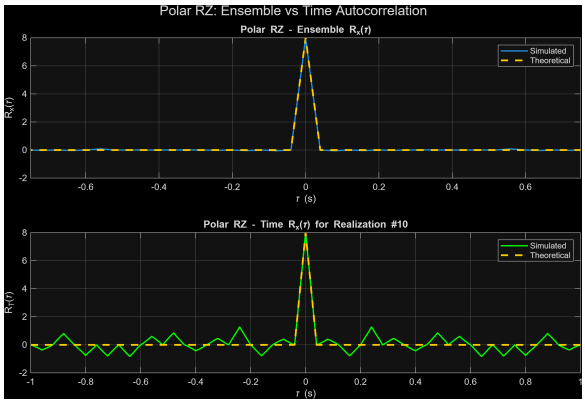

Figure 10.3: *POLAR RZ Autocorrelation.*

- In Figure 10.1, the autocorrelation value across realizations is exactly as we expected from Equation 10.2, as if we put $\tau = 0$ we get $R(0) = \frac{A^2}{4} + \frac{A^2}{4} = \frac{A^2}{2} = 8V^2$
- In Figure 10.2, the autocorrelation value across realizations is exactly as we expected from Equation 10.1, as if we put $\tau = 0$ we get $R(0) = A^2 = 16V^2$
- In Figure 10.3, the autocorrelation value across realizations is exactly as we expected from Equation 10.11, as if we put $\tau = 0$ we get $R(0) = \frac{A^2}{2} = 8V^2$

We can further verify these simulated $R(0)$ values mathematically by calculating the expected value of the squared signal, $E[X(t)^2]$, which represents the average power of the process:

- **Unipolar NRZ:**

$$E[X(t)^2] = A^2P(X(t) = A) + 0^2P(X(t) = 0) = A^2(0.5) + 0 = \frac{A^2}{2} = 8V^2 \tag{10.12}$$

- **Polar NRZ:**

$$E[X(t)^2] = A^2P(X(t) = A)+(-A)^2P(X(t) = -A) = A^2(0.5)+A^2(0.5) = A^2 = 16V^2 \tag{10.13}$$

- **Polar RZ:**

$$E[X(t)^2] = A^2(0.25) + (-A)^2(0.25) + 0^2(0.5) = \frac{A^2}{4} + \frac{A^2}{4} = \frac{A^2}{2} = 8V^2 \tag{10.14}$$

We can observe from Figure 10.1, Figure 10.2 and Figure 10.3 that the autocorrelation across time is not exactly the same but the reasoning of this is that MATLAB functions with discrete time samples so to integrate across all time for the signal we need to have infinite samples but we can say approximately that they are equal.

11

Q5 Is the Random Process Ergodic

To determine if a random process is ergodic, we need to satisfy the following conditions as learned in [4], statistics across time are the same as their statistics across the ensemble, which means:

1. $$m(t) = E(X(t)) = \langle X(t) \rangle \tag{11.1}$$

2. $$R_x(\tau) = E[X(t)X(t+\tau)] = \langle X(t)X(t+\tau) \rangle \tag{11.2}$$

   Where E is the expectation across the ensemble and $\langle \cdot \rangle$ is the time average.
   We can use the following equations to get the time average of the mean and autocorrelation

$$\langle x(t) \rangle = \lim_{T \to \infty} \frac{1}{2T} \int_{-T}^{T} x(t)\, dt \tag{11.3}$$

$$\langle x(t)x(t+\tau) \rangle = \lim_{T \to \infty} \frac{1}{2T} \int_{-T}^{T} x(t)x(t+\tau)\, dt \tag{11.4}$$

We have already calculated the mean and autocorrelation across the ensemble in the previous questions, we also graphed across time and acroess realizations, the two conditions in Equation 11.1 and Equation 11.2 are satisfied for all three formats as observed from Figures Figure 8.1, Figure 8.2, Figure 8.3, Figure 10.1, Figure 10.2, and Figure 10.3, which means that the random process is ergodic.

12

Plotting the PSD of the Ensemble

To plot the Power Spectral Density (PSD) of the ensemble, we first created a function that can do the same things as this equation we were taught in [4]:

$$S_X(f) = \mathcal{F}\{R_X(\tau)\} \tag{12.1}$$

The theoretical PSDs for each line code are derived as:

$$S_x(f)\big|_{\text{Polar NRZ}} = A^2 T \operatorname{sinc}^2(\pi f T) \tag{12.2}$$

$$S_x(f)\big|_{\text{Unipolar NRZ}} = \frac{A^2 T}{4} \operatorname{sinc}^2(\pi f T) + \frac{A^2}{4}\delta(f) \tag{12.3}$$

$$S_x(f)\big|_{\text{Polar RZ}} = \frac{A^2 T}{4} \operatorname{sinc}^2\left(\frac{\pi f T}{2}\right) \tag{12.4}$$

Listing 16: *PSD Calculation Function*

```
function [PSD_sim, PSD_theo] = compute_psd(Rx, N_fft, center, half_len, freq, Fs, A, Tb, type)
Ts  = 1/Fs;
df  = Fs / N_fft;
f   = freq;

% Simulated PSD via FFT of zero-padded Rx
Rx_padded = zeros(1, N_fft);
Rx_padded(center-half_len : center+half_len) = Rx;
PSD_sim = fftshift(abs(fft(ifftshift(Rx_padded)))) * Ts;
end
```

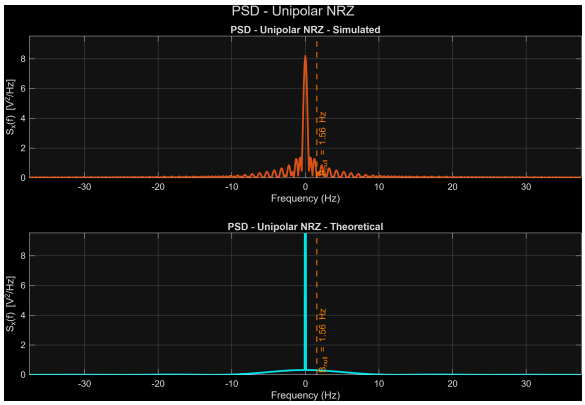

Figure 12.1: *Unipolar NRZ PSD.*

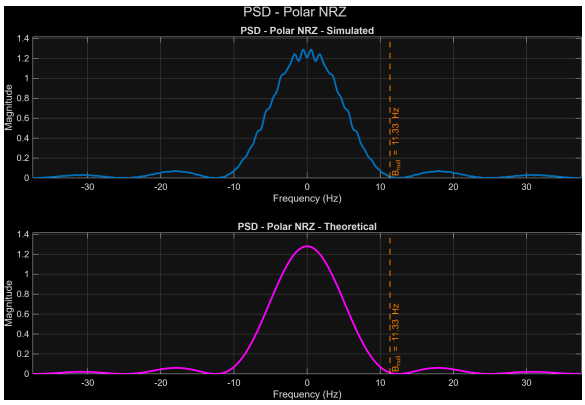

Figure 12.2: *Polar NRZ PSD.*

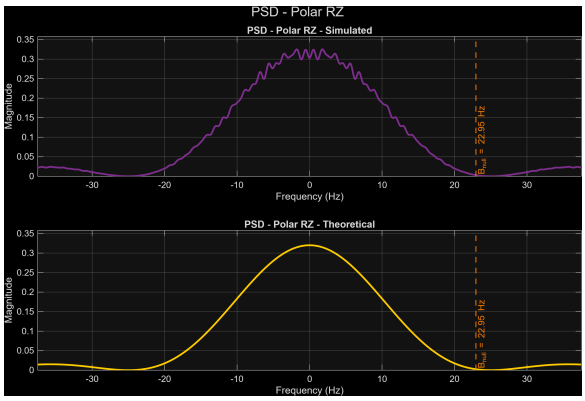

Figure 12.3: *Polar RZ PSD.*

Observations from each graph:

- **Unipolar NRZ PSD**:
  - The simulated PSD closely follows the theoretical sinc^2 envelope of Equation 12.3, confirming the accuracy of our calculations.
  - The PSD has a DC spike at $f = 0$, which is expected from the $\frac{A^2}{4}\,\delta(f)$ term in Equation 12.3, since the signal has a non-zero mean.
- **Polar NRZ PSD**:
  - The simulated PSD closely follows the theoretical sinc^2 envelope of Equation 12.2, confirming the accuracy of our calculations.
  - The PSD has no DC spike, which is consistent with Equation 12.2 containing no $\delta(f)$ term, since the signal has a zero mean.
- **Polar RZ PSD**:
  - The simulated PSD closely follows the theoretical sinc^2 envelope of Equation 12.4, confirming the accuracy of our calculations.
  - Similarly to Polar NRZ, the PSD has no DC spike since the signal is polar with a zero mean. However, the wider sinc argument $(\pi f T/2)$ in Equation 12.4 results in a wider mainlobe, placing the first null at twice the frequency compared to NRZ.

13

Q6 Signal Bandwidth

To determine the bandwidth of the signal, we use the theoretical PSDs derived in Chapter 12. The bandwidth is defined as the frequency range up to the first spectral null, since all three PSDs take a sinc^2 shape whose first null marks the edge of the mainlobe.

First null condition — sinc goes to zero when its argument is an integer:

$$\text{sinc}(x) = 0 \quad \text{when} \quad x = \pm 1, \pm 2, \ldots$$

Polar / Unipolar NRZ — from Equation 12.2 and Equation 12.3, both share the same sinc argument $\pi f T$:

$$\text{sinc}^2(\pi f T) = 0 \quad \Rightarrow \quad \pi f T = \pi \quad \Rightarrow \quad f \cdot T_b = 1$$

$$\boxed{B_{\text{null}}^{\text{NRZ}} = \frac{1}{T_b}}$$

Polar RZ — from Equation 12.4, the sinc argument is $\pi f T/2$:

$$\text{sinc}^2\left(\frac{\pi f T}{2}\right) = 0 \quad \Rightarrow \quad \frac{\pi f T}{2} = \pi \quad \Rightarrow \quad f \cdot \frac{T_b}{2} = 1$$

$$\boxed{B_{\text{null}}^{\text{RZ}} = \frac{2}{T_b}}$$

Substituting $T_b = L \times T_s = 8 \times 0.01 = 0.08$ s:

$$B_{\text{null}}^{\text{NRZ}} = \frac{1}{0.08} = 12.5 \text{ Hz}, \qquad B_{\text{null}}^{\text{RZ}} = \frac{2}{0.08} = 25 \text{ Hz}$$

Now we can make a function that calculates the bandwidth of the signal based on the first null of the sinc function and compare it with our results to see if they match.

Listing 17: *Bandwidth Calculation Function*

```
function B_null = find_first_null(freq, PSD)
pos       = freq >= 0; % Only consider non-negative frequencies for null detection.
f_pos     = freq(pos); % Corresponding frequencies.
PSD_pos   = PSD(pos);  % PSD values at non-negative frequencies.
threshold = max(PSD_pos) * 0.01; % Threshold set to 1% of max PSD to find first null.
idx       = find(PSD_pos < threshold & f_pos > 0, 1, 'first');
if ~isempty(idx)
    B_null = f_pos(idx);
else
    B_null = NaN;
end
end
```

The function `find_first_null` analyzes the PSD data to find the first frequency where the PSD drops below 1% of its maximum value, identifying the first null point which corresponds to the signal bandwidth.

As we can observe from the results in Figure 12.1, Figure 12.2, and Figure 12.3, the first nulls of the PSDs for the Unipolar NRZ and Polar NRZ occur at approximately 12.5 Hz, and for Polar RZ at 25 Hz, which matches our theoretical calculations. This confirms that our theoretical analysis of the signal bandwidth based on the first null of the sinc function is consistent with the simulated results.

Appendices

A

Appendix

```
clc; clear; close all;
%% Control Flags
N_bits         = 100;           % Number of bits per realization.
N_realizations = 500;           % Number of realizations.
Pw             = 0.08;          % Pulse width (changed to 0.08 for even L=8).
Ts             = 0.01;          % Time sample.
L              = round(Pw/Ts);  % Samples per symbol period (L=8).
A              = 4;             % Amplitude.
N_fft          = 1024;          % FFT size.
Fs             = 1/Ts;          % Sampling frequency.
freq           = (-N_fft/2:N_fft/2-1) * (Fs/N_fft);  % Frequency axis.
center         = floor(N_fft/2) + 1;                  % Center bin.
half_len       = 100;           % Max tau used in correlation_manual.

%% Data Generation
data = randi([0 1], N_realizations, N_bits);

%% Unipolar NRZ
fprintf('\nUnipolar NRZ\n');
pulse_unipolar  = ones(1, L);
ylim_unipolar   = [-1, A+1];
color_unipolar  = [0.8500 0.3250 0.0980];

ensample_unipolar = build_ensemble(data, A, pulse_unipolar, N_realizations, L, 'unipolar');

[Ex_unipolar, Ux_unipolar] = compute_means(ensample_unipolar, A/2);

[Rx_unipolar, tau_unipolar]            = correlation_manual(ensample_unipolar,       100,
↪  'ensemble');
[Rx_time_unipolar, lags_time_unipolar] = correlation_manual(ensample_unipolar(10,:), 100,
↪  'time');

plot_realizations(ensample_unipolar, 'Unipolar NRZ', ylim_unipolar);
plot_autocorr(tau_unipolar, Rx_unipolar, ...
    lags_time_unipolar, Rx_time_unipolar, 'Unipolar NRZ', Ts, A, L, 'unipolar');
plot_means(Ex_unipolar, Ux_unipolar, 'Unipolar NRZ', ylim_unipolar, A/2);
plot_stationarity(ensample_unipolar, Ts, 'Unipolar NRZ');
[PSD_sim_unipolar, PSD_theo_unipolar] = compute_psd(Rx_unipolar, N_fft, center, half_len,
↪  freq, Fs, A, Pw, 'unipolar');
```

```
plot_psd(freq, PSD_sim_unipolar, PSD_theo_unipolar, color_unipolar, 'PSD - Unipolar NRZ', 'S_x(f) [V^2/Hz]', Pw, 'unipolar');

%% Polar NRZ
fprintf('\nPolar NRZ\n');
pulse_polar_NRZ = ones(1, L);
ylim_polar_NRZ  = [-A-1, A+1];
color_polar_NRZ = [0 0.4470 0.7410];

ensample_polar_NRZ = build_ensemble(data, A, pulse_polar_NRZ, N_realizations, L, 'polar');

[Ex_polar_NRZ, Ux_polar_NRZ] = compute_means(ensample_polar_NRZ, 0);

[Rx_polar_NRZ, tau_polar_NRZ]            = correlation_manual(ensample_polar_NRZ,       100, 'ensemble');
[Rx_time_polar_NRZ, lags_time_polar_NRZ] = correlation_manual(ensample_polar_NRZ(10,:), 100, 'time');

plot_realizations(ensample_polar_NRZ, 'Polar NRZ', ylim_polar_NRZ);
plot_autocorr(tau_polar_NRZ, Rx_polar_NRZ, ...
    lags_time_polar_NRZ, Rx_time_polar_NRZ, 'Polar NRZ', Ts, A, L, 'polar_nrz');
plot_means(Ex_polar_NRZ, Ux_polar_NRZ, 'Polar NRZ', ylim_polar_NRZ, 0);
plot_stationarity(ensample_polar_NRZ, Ts, 'Polar NRZ');
[PSD_sim_polar_NRZ, PSD_theo_polar_NRZ] = compute_psd(Rx_polar_NRZ, N_fft, center, half_len, freq, Fs, A, Pw, 'polar_nrz');
plot_psd(freq, PSD_sim_polar_NRZ, PSD_theo_polar_NRZ, color_polar_NRZ, 'PSD - Polar NRZ', 'Magnitude', Pw, 'polar_nrz');

%% Polar RZ
fprintf('\nPolar RZ\n');
pulse_RZ = [ones(1, L/2), zeros(1, L/2)];
ylim_RZ  = [-A-1, A+1];
color_RZ = [0.4940 0.1840 0.5560];

ensample_RZ = build_ensemble(data, A, pulse_RZ, N_realizations, L, 'polar');

[Ex_RZ, Ux_RZ] = compute_means(ensample_RZ, 0);

[Rx_RZ, tau_RZ]            = correlation_manual(ensample_RZ,       100, 'ensemble');
[Rx_time_RZ, lags_time_RZ] = correlation_manual(ensample_RZ(10,:), 100, 'time');

plot_realizations(ensample_RZ, 'Polar RZ', ylim_RZ);
plot_autocorr(tau_RZ, Rx_RZ, lags_time_RZ, Rx_time_RZ, 'Polar RZ', Ts, A, L, 'polar_rz');
plot_means(Ex_RZ, Ux_RZ, 'Polar RZ', [-1, 1], 0);
plot_stationarity(ensample_RZ, Ts, 'Polar RZ');
[PSD_sim_RZ, PSD_theo_RZ] = compute_psd(Rx_RZ, N_fft, center, half_len, freq, Fs, A, Pw, 'polar_rz');
plot_psd(freq, PSD_sim_RZ, PSD_theo_RZ, color_RZ, 'PSD - Polar RZ', 'Magnitude', Pw, 'polar_rz');

%% Functions

```

```
function plot_realizations(ensample, name, ylim_range)
figure;
subplot(2,1,1);
stairs(ensample(1, 1:140), 'LineWidth', 1.5);
title([name, ' - Realization #1']); ylim(ylim_range); grid on; ylabel('Voltage (V)');
subplot(2,1,2);
stairs(ensample(2, 1:140), 'LineWidth', 1.5, 'Color', 'r');
title([name, ' - Realization #2']); ylim(ylim_range); grid on;
ylabel('Voltage (V)'); xlabel('Sample Index');
sgtitle([name, ' Realizations']);
end

function plot_autocorr(tau, Rx_ens, lags_t, Rx_t, name, Ts, A, L, type)
Tb      = L * Ts;
tau_sec = tau    * Ts;
lag_sec = lags_t * Ts;


switch type
    case 'polar_nrz', theo_color = [1 0 1]; % Magenta
    case 'unipolar',  theo_color = [0 1 1]; % Cyan
    case 'polar_rz',  theo_color = [1 0.8 0]; % Gold/Yellow
end

switch type
    case 'polar_nrz'
        Rx_theo_ens  = A^2 * max(0, 1 - abs(tau_sec)/Tb);
        Rx_theo_time = A^2 * max(0, 1 - abs(lag_sec)/Tb);
    case 'unipolar'
        Rx_theo_ens  = (A^2/4) * max(0, 1 - abs(tau_sec)/Tb) + (A/2)^2;
        Rx_theo_time = (A^2/4) * max(0, 1 - abs(lag_sec)/Tb) + (A/2)^2;
    case 'polar_rz'
        Rx_theo_ens  = (A^2/2) * max(0, 1 - abs(tau_sec)/(Tb/2));
        Rx_theo_time = (A^2/2) * max(0, 1 - abs(lag_sec)/(Tb/2));
end

figure;
subplot(2,1,1);
plot(tau*Ts, Rx_ens, 'LineWidth', 1.5); hold on;
plot(tau_sec, Rx_theo_ens, '--', 'Color', theo_color, 'LineWidth', 2);
title([name, ' - Ensemble R_x(\tau)']); grid on;
xlabel('\tau (s)'); ylabel('R_x(\tau)'); xlim([-0.75 0.75]);
legend('Simulated', 'Theoretical');

subplot(2,1,2);
plot(lags_t*Ts, Rx_t, 'g', 'LineWidth', 1.5); hold on;
plot(lag_sec, Rx_theo_time, '--', 'Color', theo_color, 'LineWidth', 2);
title([name, ' - Time R_x(\tau) for Realization #10']); grid on;
xlabel('\tau (s)'); ylabel('R_T(\tau)');
legend('Simulated', 'Theoretical');
sgtitle([name, ': Ensemble vs Time Autocorrelation']);
end
```

```
function plot_means(Ex, Ux, name, ylim_range, theo_mean)

contrast_mean = [1 0.2 0.2]; % Bright Coral Red

figure;
subplot(2,1,1);
plot(Ex); hold on;
yline(theo_mean, '--', 'Color', contrast_mean, 'LineWidth', 2);
ylim(ylim_range); grid on;
title([name, ' - E[X(t)]']); xlabel('Sample Index'); ylabel('Mean (V)');
legend('Simulated', 'Theoretical');

subplot(2,1,2);
plot(Ux); hold on;
yline(theo_mean, '--', 'Color', contrast_mean, 'LineWidth', 2);
ylim(ylim_range); grid on;
title([name, ' - <X(t)> per Realization']); xlabel('Realization'); ylabel('Mean (V)');
legend('Simulated', 'Theoretical');
sgtitle([name, ': E[x(t)] vs <x(t)>']);
end

function plot_stationarity(ensample, Ts, name)
t1 = 10; t2 = 50; window_size = 350;
[R_t1, lags_t1] = correlation_manual(ensample(:, t1:t1+window_size), 50, 'ensemble');
[R_t2, lags_t2] = correlation_manual(ensample(:, t2:t2+window_size), 50, 'ensemble');
figure;
plot(lags_t1*Ts, R_t1, 'b', 'LineWidth', 1.5); hold on;
plot(lags_t2*Ts, R_t2, 'r--', 'LineWidth', 1.5);
grid on; legend('R_x at t_1', 'R_x at t_2');
title([name, ' - Stationarity Check']); xlabel('\tau (s)'); ylabel('R_x(\tau)');
end

function [PSD_sim, PSD_theo] = compute_psd(Rx, N_fft, center, half_len, freq, Fs, A, Tb, type)
Ts  = 1/Fs;
df  = Fs / N_fft;
f   = freq;

% Simulated PSD via FFT of zero-padded Rx
Rx_padded = zeros(1, N_fft);
Rx_padded(center-half_len : center+half_len) = Rx;
PSD_sim = fftshift(abs(fft(ifftshift(Rx_padded)))) * Ts;

% Theoretical PSD
f_eps = f + 1e-12;
switch type
    case 'polar_nrz'
        PSD_theo = (A^2 * Tb) * (sinc(f_eps * Tb)).^2;
    case 'unipolar'
        PSD_theo = (A^2 * Tb / 4) * (sinc(f_eps * Tb)).^2;
        [~, dc_idx] = min(abs(f));
        PSD_theo(dc_idx) = PSD_theo(dc_idx) + (A^2/4) / df;
```

```
    case 'polar_rz'
        PSD_theo = (A^2 * Tb / 4) * (sinc(f_eps * (Tb/2))).^2;
end
end

function plot_psd(freq, PSD_sim, PSD_theo, color, title_str, ylabel_str, Tb, type)

switch type
    case 'polar_nrz', theo_color = [1 0 1];
    case 'unipolar',  theo_color = [0 0.9 0.9];
    case 'polar_rz',  theo_color = [1 0.8 0];
end


[~, dc_idx] = min(abs(freq));
mask = true(size(PSD_sim));
mask(max(1, dc_idx-1):min(length(freq), dc_idx+1)) = false;
if strcmp(type, 'unipolar')
    y_max = max(PSD_sim(mask)) * 1.5;
else
    y_max = max(PSD_sim) * 1.1;
end

B_null = find_first_null(freq, PSD_sim);

figure;

subplot(2,1,1);
plot(freq, PSD_sim, 'LineWidth', 2, 'Color', color); hold on;
if ~isnan(B_null)
    xline(B_null, '--', 'Color', [1 0.5 0], 'LineWidth', 1.5, ...
        'Label', sprintf('B_{null} = %.2f Hz', B_null), ...
        'LabelVerticalAlignment', 'bottom');
end
grid on; title([title_str, ' - Simulated']);
xlabel('Frequency (Hz)'); ylabel(ylabel_str);
xlim([-3/Tb, 3/Tb]); ylim([0, y_max]);

subplot(2,1,2);
plot(freq, PSD_theo, 'LineWidth', 2, 'Color', theo_color); hold on;
if ~isnan(B_null)
    xline(B_null, '--', 'Color', [1 0.5 0], 'LineWidth', 1.5, ...
        'Label', sprintf('B_{null} = %.2f Hz', B_null), ...
        'LabelVerticalAlignment', 'bottom');
end
grid on; title([title_str, ' - Theoretical']);
xlabel('Frequency (Hz)'); ylabel(ylabel_str);
xlim([-3/Tb, 3/Tb]); ylim([0, y_max]);

sgtitle(title_str);
end

```

```
function B_null = find_first_null(freq, PSD)
pos      = freq >= 0; % Only consider non-negative frequencies for null detection.
f_pos    = freq(pos); % Corresponding frequencies.
PSD_pos  = PSD(pos); % PSD values at non-negative frequencies.
threshold = max(PSD_pos) * 0.01; % Threshold set to 1% of max PSD to find first null.
idx      = find(PSD_pos < threshold & f_pos > 0, 1, 'first');
if ~isempty(idx)
    B_null = f_pos(idx);
else
    B_null = NaN;
end
end

function [Ex, Ux] = compute_means(ensample, theoretical_mean)
Ex = manual_mean(ensample, 1);   % Ensemble mean (across realizations).
Ux = manual_mean(ensample, 2);   % Time mean (across samples).
fprintf('  Sample 10 ensemble mean  = %.4f V (Theoretical: %.2f V)\n',
↪  manual_mean(ensample(:,10)),  theoretical_mean);
fprintf('  Realization 15 time mean = %.4f V (Theoretical: %.2f V)\n',
↪  manual_mean(ensample(15,:)), theoretical_mean);
end

function ensample = build_ensemble(data, A, pulse, N_realizations, L, type)
if strcmp(type, 'unipolar'), Tx = data * A;           % Map bits to {0, A}.
else,                        Tx = (2*data - 1) * A;   % Map bits to {-A, +A}.
end
ensample = kron(Tx, pulse);                           % Upsample with pulse shape.
Td = randi([0, L-1], N_realizations, 1);              % Random delay per realization.
for i = 1:N_realizations
    ensample(i,:) = circshift(ensample(i,:), [0, Td(i)]);
end
end

function mean_val = manual_mean(data, dim)
if nargin == 1,    mean_val = sum(data, 'all') / numel(data);  % All elements.
elseif dim == 1,   mean_val = sum(data, 1) / size(data, 1);    % Column-wise (ensemble).
elseif dim == 2,   mean_val = sum(data, 2) / size(data, 2);    % Row-wise (time).
end
end

function [Rx, tau] = correlation_manual(signal, max_tau, type)
tau = -max_tau:max_tau;      % Lag vector.
Rx  = zeros(1, length(tau)); % Output array.
if strcmp(type, 'ensemble')
    [~, N_samples] = size(signal);
    for i = 1:length(tau)
        shift = tau(i);
        if shift >= 0
            Rx(i) = manual_mean(signal(:, 1:N_samples-shift) .* signal(:, 1+shift:N_samples));
        else
            Rx(i) = manual_mean(signal(:, 1-shift:N_samples) .* signal(:, 1:N_samples+shift));
        end
```

```
    end
elseif strcmp(type, 'time')
    if isvector(signal), signal = signal(:)'; end  % Ensure row vector.
    N = length(signal);
    for i = 1:length(tau)
        lag = tau(i);
        if lag >= 0
            Rx(i) = manual_mean(signal(1:N-lag) .* signal(1+lag:N));
        else
            Rx(i) = manual_mean(signal(1-lag:N) .* signal(1:N+lag));
        end
    end
end
end
```

References

[1] D. M. Nafie, *Digital communications lecture 2*, Mar. 2026.

[2] D. M. Nafie, *Digital communications lecture 3*, Mar. 2026.

[3] E. M. Khalid, *Section 4 - digital communications*, Mar. 2026.

[4] D. M. Nafie, *Digital communciations lecture 4*, Mar. 2026.